

DeepSeek-V3 Fused MoE - a BitsEvolve case study

Qasim Khan
qasim.khan@datadoghq.com
Datadog
New York, USA

Jai Menon
jai.menon@datadoghq.com
Datadog
New York, USA

1 Overview

1.1 BitsEvolve

We used **BitsEvolve**, an LLM-guided evolutionary search and optimization framework developed within Datadog, to explore the contest starter code and iterate on kernel candidates. BitsEvolve is designed for systems and code optimization across compute-intensive hotspots, ML kernels, configuration tuning, and related optimization problems. For this contest, the starting point was the reference code provided with the starter kit and the base GPU-specific optimization workflow in BitsEvolve. Candidate kernels were evaluated against the contest workload traces using the `flashinfer-bench` correctness and performance harness on NVIDIA B200 hardware. In addition to measured B200 feedback, the search context included Blackwell idioms from CUTLASS/CuTe examples, NVIDIA PTX/SM100 documentation, and related high-performance kernel references.

1.2 Kernel summary

The submitted kernel targets the contest’s FP8 block-scaled, top-8 routed MoE definition on NVIDIA B200 (SM100a). It is a single Python module that JIT-compile a custom CUDA extension through PyTorch using CUDA 13.2 and `-arch=sm_100a`. CUTLASS 3.7 and CuTe headers are inlined into the extension and used as building blocks for grouped-GEMM and block-scaled MMA paths; the runtime path does not call a prebuilt kernel binary such as FlashInfer, DeepGEMM, or cuBLAS.

Figure 1 shows the implemented DeepSeek-V3 MoE forward step. The pipeline runs in two execution modes selected by T . For $T \leq 2048$, a fixed-shape graph-safe path uses pre-allocated workspace and CUDA-graph replay so each timed call collapses to one graph submission. For $T > 2048$, a compact dynamic path processes only locally valid assignments. Within both modes the kernel mix is warp-specialized routing, dispatch, gather, SwiGLU+FP8-requant kernels; CUTLASS 3.7 SM100 grouped-GEMM with an AB-swapped small- M tile and blockwise tile configs selected by dispatched-token count; a hardware block-scaled Mx8 grouped-GEMM for $T \geq 4096$; routing-weight fold into the GEMM2 A -scale; and a two-kernel reduce-scatter via 2D inverse permutation.

1.3 Workload results

Measured on Modal B200 via the contest harness at `atol=1`, `rtol=0.3`, and `required_matched_ratio=0.9`, all 19 workloads pass correctness. The harness reports speedup against the simple PyTorch reference kernel that defines correctness; that is the comparison baseline used throughout this write-up, not FlashInfer. Against the FlashInfer kernel used by the contest benchmark, the final submitted kernel was 1.68 $\times$ faster overall.

Table 1: Aggregate speedup over all 19 workloads versus the reference kernel.

Metric	Value
Arithmetic mean	73.15 $\times$
Geometric mean	70.19 $\times$
Harmonic mean	67.88 $\times$
Min / max	48.12 $\times$ / 144.91 $\times$

2 Pipeline Architecture

For a batch of T tokens with hidden size $K = 7168$, $E = 32$ local experts, intermediate size $I = 2048$, and DeepSeek-V3 grouped routing (`top_k=8`, `n_group=8`, `topk_group=4`), the contest definition is the standard fused MoE forward: route each token to top- k experts, dispatch local assignments, gather FP8 hidden states and scales, run GEMM1, apply SwiGLU and FP8 requantization, run GEMM2, and reduce-scatter the routed partial outputs into a single BF16 output per token. Steps after routing operate on assignments rather than original tokens, so the dispatched length is `total_valid  $\leq T \cdot \text{top}_k$` . Most design effort goes into ensuring the kernel never materializes $T \cdot \text{top}_k$ worth of compute or memory traffic when it can be avoided.

Figure 2 summarizes the regime selection. Both paths share one workspace keyed by shape and device. The workspace holds routing tables, dispatch metadata, packed activations and scales, GEMM outputs, expert pointer arrays, Mx8 SFA/SFB layouts, per-row FP8 requant scales, BF16 output, and CUTLASS workspace. Activation-derived contents are recomputed on every call. The only persistent state is the transcoded Mx8 weight tensor and its pre-packed SFB layout, a deterministic function of input weight pointers computed lazily on first use. The CUDA-graph cache is keyed by T , device, and input data pointers; for the contest harness, fixed tensors across timing iterations make every iteration after warmup one graph replay.

3 Kernel Design and Optimizations

3.1 Routing

DeepSeek-V3 routing over 256 global experts is grouped into eight groups of 32, with four active groups and top-8 experts. A naive implementation needs separate passes for sigmoid and bias, group scoring, and expert top- k , plus sorts and intermediate writes. We do all three in one single-warp kernel launched as `<<<T, 32>>>`, with no shared memory and no `__syncthreads`. Each lane owns eight experts; per-group top-2, top-4-of-8 groups, and final top-8 are implemented as warp reductions using shuffle and ballot operations. The original 256-thread block version showed 41.7% barrier stalls in NCU at $T = 14107$ and took roughly 100 μs . The single-warp



Figure 1: Seven-stage fused MoE forward pass.

Regime	Implementation path
$T \leq 2048$	Graph-safe path: fixed shape $T \cdot \text{top_k}$ and CUDA-graph replay. Uses blockwise FP8 GEMM with software-scaled mainloop.
$T > 2048$	Dynamic path: compact <code>total_valid</code> rows and skip non-local assignments.
$T \geq 4096$	MxF8 sub-path inside the dynamic path: <code>tcgen05.mma.kind.mxf8f6f4</code> hardware block-scaled GEMM.

Figure 2: Execution regimes selected by total dispatched tokens T .

rewrite reduces routing to about $88 \mu\text{s}$ and drops barrier stalls to about 4%.

3.2 Dispatch

Dispatch turns the $[T, \text{top_k}]$ expert-id and weight table into per-expert counts, exclusive-scan offsets, expert-sorted token ids and weights, GEMM problem-size triplets, and the inverse 2D permutation used by reduce-scatter. We use one CTA that loops the $T \cdot \text{top_k}$ assignments, performs atomic per-expert counts, runs a warp scan over local expert counts, and re-traverses to fill sorted token ids. A one-CTA-per-expert decomposition would parallelize count and scan but would turn the exclusive scan into a global-memory atomic chain; per-CTA work is too small to amortize launch overhead, and the single-CTA kernel already runs in about $10 \mu\text{s}$ at $T = 14107$.

3.3 Gather

GEMM1 consumes activations as a row-major $[\text{total_valid}, K]$ packed matrix with per-row scale blocks. The standard gather streams 7168 FP8 elements and 56 FP32 scales per row. It detects whether the contest scale tensor is laid out as $[T, K/128]$ or $[K/128, T]$ using stride checks, avoiding a `contiguous()` call that would break graph capture. The MxF8 gather also transcodes FP32 scales to UE8M0, folds scale sign into the FP8 payload, and writes UE8M0 bytes directly into CUTLASS’s tiled SFA layout via pre-computed offsets. This removes a standalone transcode and SFA-pack pass. The previous MxF8 gather ran about $99 \mu\text{s}$ at $T = 14107$ with 48.5% barrier stalls; the single-warp version is about $90 \mu\text{s}$.

3.4 GEMM1 and GEMM2

The GEMMs are CUTLASS 3.7 grouped-GEMM instances. The custom part is a tile-config classifier that selects one of four configurations by dispatched-token count, including an AB-swap trick that reuses a small- M tile for regimes where per-expert M is often 1–8. A targeted sweep showed $T = 901$ prefers a smaller `tile_M=64` configuration, while $T = 52$ – 80 prefer the AB-swapped small- M path.

For $T \geq 4096$, we replace blockwise grouped GEMM with the prepacked MxF8 grouped-GEMM path, which uses SM100’s hardware block-scaled MxF8 MMA instruction. The key difference is where the scale multiply happens: blockwise FP8 applies FP32 scales in software in the epilogue, while MxF8 applies UE8M0 scales inside the tensor core. Pure GEMM at $T = 14107$ is $472 \mu\text{s}$ with MxF8 versus $575 \mu\text{s}$ with blockwise FP8, an 18% GEMM speedup. End-to-end at $T = 14107$, the shipped MxF8 path is 1.195 ms versus 1.263 ms for blockwise FP8, a 5–6% wall-clock win.

Three details make MxF8 work under contest semantics. First, UE8M0 is unsigned, so the FP32-scale sign must be folded into the FP8 payload; otherwise match ratio drops to roughly 55%. Second, UE8M0 transcode uses `ceil-to-pow-2` of $\log_2$; rounding up keeps residual error inside tolerance over $K = 7168$. Third, SFA/SFB byte layouts must match the MMA tile, with byte offsets pre-computed on device. Activation-side transcode and SFA-pack are fused into gather for GEMM1 and SwiGLU for GEMM2; weight-side transcoded tensors and pre-packed SFB layouts persist in workspace keyed by immutable input weight pointers.

3.5 SwiGLU, Routing-Weight Fold, and Reduce-Scatter

GEMM1 emits $[M, 2H]$ BF16 with $H = 2048$. The MxF8 SwiGLU kernel computes $\text{SiLU}(\text{gate}) \cdot \text{up}$, multiplies by the routing weight, reduces a per-row `absmax` via warp shuffle, requantizes to FP8, and writes the UE8M0 row scale directly into GEMM2’s SFA layout. The original block-level MxF8 SwiGLU took $59 \mu\text{s}$ at $T = 14107$ with 40.7% barrier stalls; the single-warp rewrite takes about $52 \mu\text{s}$.

The routing-weight fold moves the multiply from reduce-scatter into the GEMM2 input scale. Instead of computing $\sum_k w_{t,k} \cdot \text{GEMM2}_{m_k}$ in the final scatter, the SwiGLU kernel multiplies each row by its routing weight before requantization. GEMM2’s A-scale absorbs this factor, and the reduce-scatter sees already weighted rows. This removes one FP32 multiply and one HBM read from the reduce-scatter inner loop and composes naturally with MxF8 SFA writing.

The final reduce-scatter is a two-kernel chain. First, an inverse-bucket kernel builds a 2D token-to-row permutation and per-token slot counts. Second, the reduce-scatter kernel launches one CTA per output token, reads up to eight row indices, accumulates GEMM2 rows in registers via vectorized loads, and writes the BF16 result directly. This removes three launches and avoids the previous $[T, \text{top_k}, K]$ intermediate, about 500 MB at $T = 14107$.

4 Performance Analysis

Each timed iteration is a CUDA-graph replay for $T \leq 2048$ or a single dynamic call for $T > 2048$, with input tensors held fixed across iterations. Table 2 partitions the 19 workloads into representative regimes. NCU instrumentation slows wall-clock by roughly 20–40%, so the percentages and bottleneck classification matter more than instrumented absolute time.

Table 2: Representative per-regime performance. Speedup is against the reference kernel.

T	Binding constraint	Latency (ms)	Speedup
1	CUDA-graph replay floor	0.074	144.91×
80	GEMM HBM-bound plus auxiliary launches	0.253	61.47×
901	Per-expert tile fill and weight bandwidth	0.340	57.37×
14107	89% MxF8 tensor-core pipe	0.925	48.12×

The speedup column reads in the opposite direction from the binding constraint: launch-bound workloads look like the biggest win because the reference pays full Python and launch overhead, while the compute-bound regime has the smallest multiplier because both kernels converge on the same HBM and MMA pipes. The largest local wins are CUDA-graph replay at tiny T , AB-swapped small- M GEMM for $T \leq 80$, compact dynamic dispatch for large T , the MxF8 hardware block-scaled GEMM path for $T \geq 4096$, and the two-kernel reduce-scatter that avoids the prior 500 MB intermediate.

The 19-workload total is approximately 5.3 ms. The remaining structural gaps are regime-specific: $T = 1$ is still far above the graph-replay floor because the chain has too many kernels; $T = 80$ retains auxiliary-launch overhead and small- M tile-fill loss; $T = 901$ is limited by the GEMM1-output HBM round trip; and $T = 14107$ is close to the modeled MxF8 compute floor, with the remaining tail in route, gather, SwiGLU, and scatter. The medium and compute-bound regimes share one architectural blocker: GEMM1 and GEMM2 are separate kernels that exchange a $[M, N]$ BF16 tensor through HBM, while SwiGLU and reduce-scatter are still launched separately. A persistent SM100a kernel that fuses the whole chain through TMEM-resident accumulators would close most of the remaining gap.

5 Human and Agent Contributions

The submission was primarily produced using BitsEvolve, Datadog’s LLM-guided evolutionary search and optimization framework. The contest-provided harnesses were integrated into a BitsEvolve workflow and candidate evaluations were performed on B200 GPUs using Modal.

During optimization, we incorporated additional hinting and feedback to improve Blackwell/B200 output quality over baseline BitsEvolve. Initial candidates were Triton-based and reached a plateau due to code-generation gaps around the SM100a block-scaled MMA type, TMEM-resident accumulators, SM100 grouped-GEMM TMA layouts, and async machinery. Additional hinting and feedback around CUTLASS/CuTe use helped move the search past this plateau.

Further into the evolutionary process, the workflow itself was refined. Code-path specialization was already a directive, but the model did not always prioritize it during iterations; tightening that instruction helped improve performance. More specific feedback around measurement noise also helped avoid unproductive trajectories: for example, a change from 1.089×

to 1.10×

Acknowledgments

We thank the MLSys 2026 FlashInfer AI Kernel Generation Contest organizers for the starter kit, workload traces, and evaluation harness, and Modal for providing compute credits used for B200 evaluations.

References

- [1] 2026. BitsEvolve: Datadog Skylab Blogpost. <https://adrs-ucb.notion.site/datadog>. Accessed 2026-05-15.
- [2] 2026. FlashInfer Bench. <https://github.com/flashinfer-ai/flashinfer-bench>. Contest harness and workload traces used for correctness and performance evaluation.
- [3] 2026. NVIDIA CUTLASS and CuTe. <https://github.com/NVIDIA/cutlass>. SM100 grouped-GEMM, CuTe tensor abstractions, epilogues, and blockwise FP8 examples.
- [4] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. <https://arxiv.org/abs/2412.19437>. *arXiv preprint arXiv:2412.19437* (2024).
- [5] NVIDIA 2026. *NVIDIA Parallel Thread Execution ISA*. NVIDIA. Blackwell additions including `tcgen05.mma.kind.mxf8f6f4.block_scale` and launch-dependency control.